

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

INTRO TO CRITICAL INFRASTRUCTURE FOR IT LEARNERS

LAB 04 — INTRODUCTION TO SUPERVISORY CONTROL AND DATA ACQUISITION (SCADA)

Student Edition

Student name	Sidra Shaikh
Registration No.	C11/26/ACIS/17349
Lab	Lab 04 — Introduction to Supervisory Control and Data Acquisition (SCADA)
Date	26 September 2026

Authorized ICDFA training environments only
icdfa.edu.ng

Lab 04: Introduction to Supervisory Control and Data Acquisition (SCADA)

Course: Intro to Critical Infrastructure for IT Learners | **Estimated Duration:** 3–4 hours

Submission deadline: As published in ICDFA E-Campus.

Authorization & Safety: Perform all activities only on the ICDFA-provided virtual machines, packet captures and isolated training networks. Do not bridge these VMs to production networks, scan external systems, or reuse credentials observed in training data.

Learning Outcomes

- Explain the purpose and architecture of SCADA systems.
- Identify the roles of PLCs/RTUs, HMI, alarms, event recording and historization.
- Start FUXA in the training SCADA VM and load a prepared HMI project.
- Connect a SCADA HMI to the OpenPLC Runtime over Modbus TCP in the isolated lab.
- Observe an automated tank-fill process and interpret operator-facing indications.

Laboratory Environment

System	Expected Address	Account	Password
PLC VM	192.168.1.10	plc	password
SCADA VM	192.168.2.10	scada	password

Note: If the instructor-provided VM image uses updated addressing or credentials, use the values published in E-Campus. Do not alter addressing unless instructed.

Pre-Lab Checklist

- Read the complete lab before beginning.
- Confirm you can access the required VM(s) and that virtualization support is enabled.
- Use the instructor-specified isolated/host-only network.
- Create or restore the clean starting snapshot for this lab.
- Prepare a report document with your name, registration number, course, lab number and date.
- Capture evidence as you progress rather than repeating the lab at the end.

Procedure

Step 1. Import/start the ICDFA PLC and SCADA VMs using the isolated network configuration supplied by your instructor. Create LAB04-START snapshots if they are not already available.

Step 2. On the SCADA VM, sign in and open a terminal. Start FUXA using the training image command: `sudo fuxa`. Enter the SCADA account password when prompted.

Step 3. Open Firefox and browse to <http://127.0.0.1:1881/editor> to access the local FUXA editor.

Step 4. Use the Save/Open Project function to load TankFillTest.json from the instructor-provided project location.

Step 5. Open FUXA project settings > Connections. Edit the openplc device and set the Modbus TCP endpoint to 192.168.1.10:502. Save the connection.

Step 6. Switch to the PLC VM. Open <http://localhost:8080/> in the local browser and sign in to OpenPLC Runtime using the credentials configured in the training image.

Step 7. Navigate to Programs, select the provided TankFillSim.st program, upload it, name it Tank Fill Sim and allow it to compile.

Step 8. Go to Dashboard and start the PLC. Confirm that the runtime status shows Running.

Step 9. Return to the SCADA VM and FUXA. Run/Play the HMI project.

Step 10. Observe the tank-fill simulation. Record the tank-level range, valve open/closed indication and any trend information presented to the operator.

Step 11. Create a simple architecture diagram in your report showing Field/Process -> PLC -> Industrial Protocol/Network -> SCADA/HMI -> Operator, and indicate where alarms/events/historian functions belong conceptually.

Step 12. Capture the required screenshots and stop the simulation cleanly when finished.

Required Evidence & Submission

- Screenshot showing OpenPLC Runtime in Running state.
- Screenshot showing the FUXA tank-fill HMI in operation.
- A labelled SCADA architecture diagram.
- A 150–250 word explanation of the difference between control logic in the PLC and supervisory visualization in the HMI/SCADA layer.

Recommended filename: <RegistrationNo>_LAB04_Introduction_to_Supervisory_Control_and_Data_Acquisition_SCADA.pdf

Knowledge Check

1. What is the primary purpose of an HMI in a SCADA system?

Response: _____

2. What is the difference between an alarm, an event record and historized process data?

Response: _____

3. Why should a SCADA operator have context rather than only raw sensor values?

Response: _____

4. What role does the PLC play compared with the SCADA application?

Response: _____

Marking Rubric — 100 Marks

Criterion	Marks	Standard
Technical execution	50	Lab procedure completed correctly; required configuration/analysis works as intended.
Evidence	20	Screenshots/files are complete, readable, relevant and clearly labelled.
Analysis & knowledge check	20	Responses demonstrate understanding of the industrial-control concepts observed.
Professional reporting	10	Clear structure, correct terminology, concise writing, file naming and submission compliance.

Submission Integrity: Evidence must come from the student's own assigned laboratory environment. Screenshots should show enough context to verify the activity. Do not submit another learner's screenshots or project files.

Solution Below:



1. Objective

This lab introduces Supervisory Control and Data Acquisition (SCADA) systems by connecting a FUXA-based SCADA HMI to an OpenPLC Runtime PLC over Modbus TCP, observing an automated tank-fill process, and analysing the relationship between PLC control logic and SCADA/HMI supervisory visualization.

2. Procedure Summary

The FUXA SCADA VM was started and the TankFillTest.json HMI project was loaded in the FUXA editor. The openplc device connection was configured to point to the PLC VM at 192.168.1.10:502 over Modbus TCP. On the PLC VM, OpenPLC Runtime was accessed at <http://localhost:8080/> and a Structured Text (.st) tank-fill simulation program was uploaded, compiled, named Tank Fill Sim, and started from the Dashboard.

2.1 Issue encountered: missing TankFillSim.st

The file named in the lab instructions, TankFillSim.st, was not present at the documented location (Documents > PLC Programing Language Examples > Tank Fill Sim) on the assigned PLC VM, and the VM's only local account was icdfa rather than the documented plc account. A filesystem-wide search located only a Beremiz-format PLCopen project (plc.xml) containing the same tank-fill-sim logic, and an unrelated non-compiling snippet (07_Tank_Fill_Sim.st) under a course examples folder. This discrepancy was reported to the instructor for confirmation.

To keep the lab progressing, the Structured Text logic embedded in the Beremiz project's plc.xml was extracted and reassembled into a standalone .st file with the required PROGRAM/VAR/CONFIGURATION structure and correctly formatted AT-located variable declarations. This reconstructed program compiled successfully in OpenPLC Runtime and was used to complete the remaining lab steps.

3. Required Evidence

3.1 OpenPLC Runtime — Running state

The Dashboard confirms Status: Running with Program: Tank Fill Sim loaded and compiled successfully.

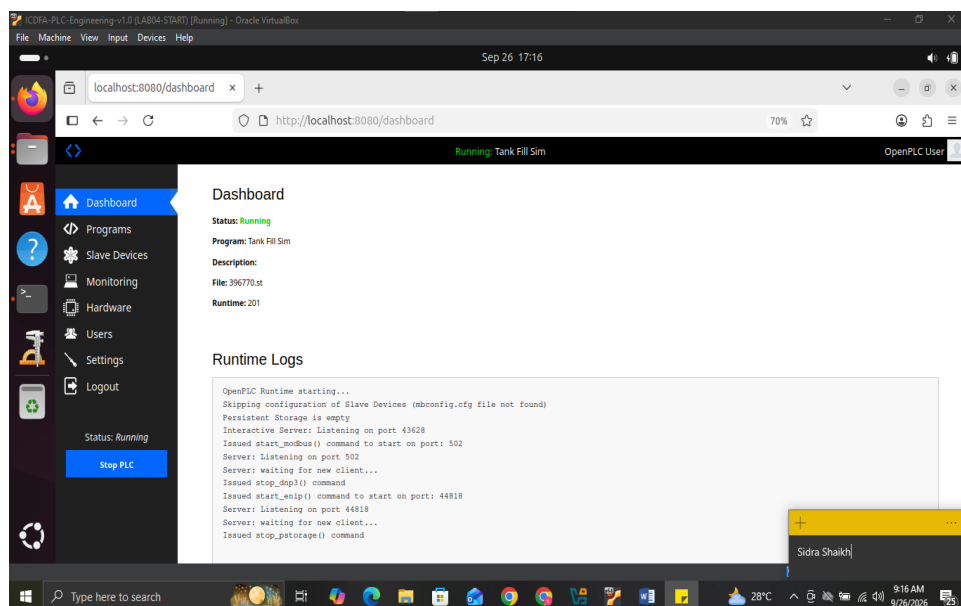


Figure 1. OpenPLC Runtime dashboard showing Running status.

3.2 FUXA tank-fill HMI in operation

The FUXA HMI loaded the Tank Fill process view and established a live Modbus TCP connection to the PLC (green Local discovery relay status, target localhost:1502), confirming SCADA-to-PLC connectivity. The process graphic displays the Fill valve (Coil 0) and Process Tank with a 0-10 ft operating range and an embedded tank-level trend panel.

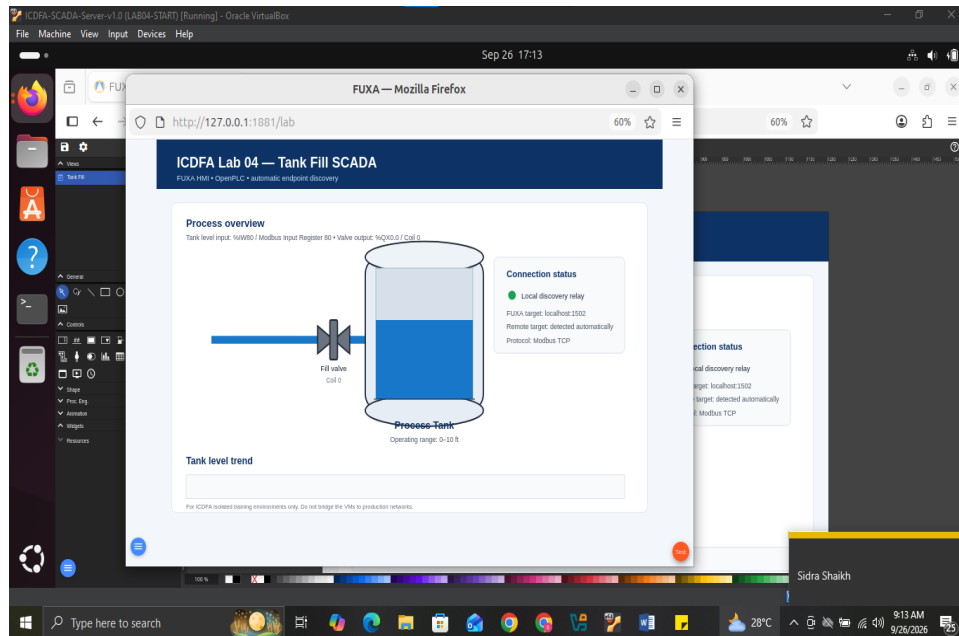


Figure 2. FUXA HMI connected to OpenPLC Runtime over Modbus TCP.

4. SCADA Architecture Diagram

The diagram below shows the conceptual data flow for this lab: the physical field process is read and actuated by the PLC, which exchanges data with the SCADA/HMI layer over an industrial protocol (Modbus TCP), which in turn presents the process to the operator. Alarms, event recording and historization are shown attached at the SCADA/HMI layer, since this is where process values are typically logged, alarmed on and pushed to a historian, even though a dedicated historian may sit on a separate server in larger deployments.

SCADA Architecture — Lab 04 Tank Fill Process

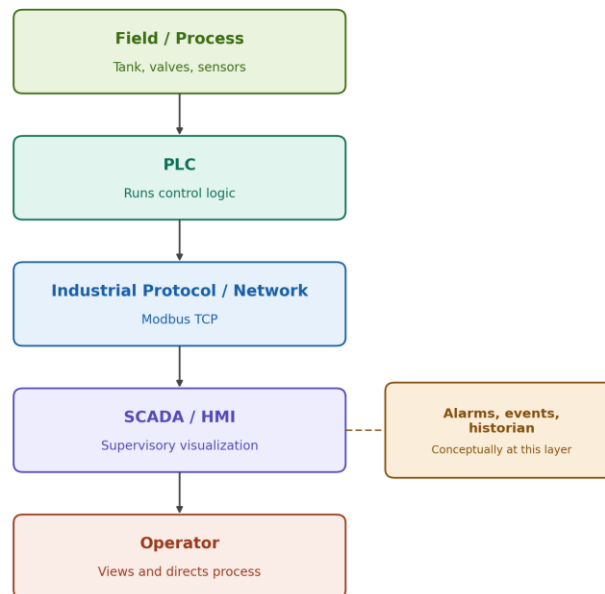


Figure 3. SCADA architecture: Field/Process -> PLC -> Industrial Protocol/Network -> SCADA/HMI -> Operator.

5. PLC Control Logic vs. HMI/SCADA Supervisory Visualization

The PLC and the SCADA/HMI layer serve fundamentally different roles in an industrial control system, even though they work together on the same process.

The PLC executes the actual control logic. It runs a scan cycle that reads inputs, evaluates programmed logic (in this lab, Structured Text), and drives outputs, deciding, moment to moment, whether a fill valve opens or a drain valve closes based on real-time conditions like tank level. This logic runs locally on the PLC regardless of whether anyone is watching; it does not depend on a SCADA connection to function correctly.

The SCADA/HMI layer, by contrast, does not control the process directly. It polls the PLC over an industrial protocol (Modbus TCP in this lab) to read current values and present them to a human operator in a readable, graphical form: tank levels, valve states, and trend history. It also provides the operator interface for supervisory commands, alarms, event logging, and historization, giving context and situational awareness rather than making control decisions itself.

In short, the PLC is where safety-critical, deterministic control logic lives, while SCADA/HMI is a supervisory and observational layer built on top of it. If the SCADA connection is lost, the PLC continues running its logic independently; if the PLC logic is faulty or absent, the SCADA layer can only display and report on the process, not fix it.

6. Knowledge Check

1. What is the primary purpose of an HMI in a SCADA system?

An HMI's primary purpose is to give a human operator a clear, readable graphical view of an industrial process, including current values (e.g. tank level, valve state) and trend history, and to provide a supervisory interface for issuing commands and viewing alarms, without itself performing the underlying control logic.

2. What is the difference between an alarm, an event record and historized process data?

An alarm is a real-time notification that a process variable has crossed a defined threshold and needs operator attention. An event record is a timestamped log entry capturing a discrete occurrence, such as an operator starting a pump or an alarm being acknowledged, used to reconstruct what happened and when. Historized process data is the continuous recording of process values over time (e.g. tank level every scan), used for trending, analysis and compliance reporting rather than for immediate alerting.

3. Why should a SCADA operator have context rather than only raw sensor values?

Raw sensor values on their own do not tell an operator whether a reading is normal, trending toward a problem, or requires action. Context, such as setpoints, alarm limits, historical trends and related process states, lets the operator interpret a value correctly and respond appropriately, reducing the risk of missed problems or unnecessary intervention.

4. What role does the PLC play compared with the SCADA application?

The PLC performs the direct, real-time control of the physical process by executing logic against field inputs and outputs, independent of any supervisory system. The SCADA application does not control the process directly; it supervises it, polling the PLC for data, visualizing it for the operator, and providing a channel for supervisory commands, alarms and historization.